What is a Software Design Pattern?

Software design patterns are reusable solutions that are not exactly the final product. It is a type of template for software design. They are not a code that can be directly converted into the software.

Take an example of different clothing styles like denim, striped shirts, or even chinos. They are all templates based on which we can create new clothes, but they are not a final product**.**

Design Pattern:

Design patterns have been evolved over a long period of time and they provide best solutions to certain problems faced during software development.

Creational Patterns
These design patterns provide a way to create objects while hiding the creation logic, rather than instantiating objects directly using new operator. This gives program more flexibility in deciding which objects need to be created for a given use case.

Structural Patterns
These design patterns concern class and object composition. Concept of inheritance is used to compose interfaces and define ways to compose objects to obtain new functionalities.

Behavioral Patterns
These design patterns are specifically concerned with communication between objects.

Problem **Given:**
Suppose you want to create a class for which only a single instance (or object) should be created and that single object can be used by all other classes.

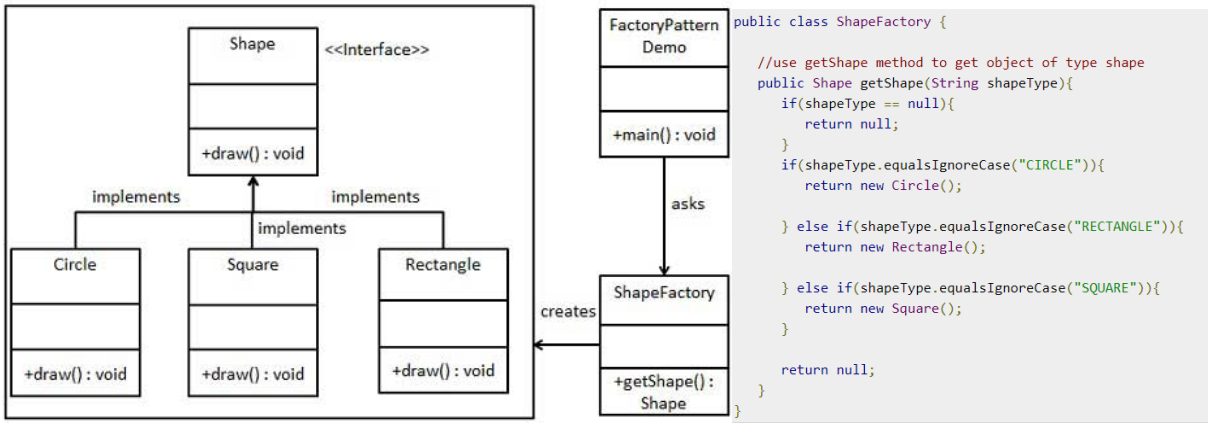
Solution:
Singleton design pattern is the best solution of above specific problem. So, every design pattern has some specification or set of rules for solving the problems. What are those specifications, you will see later in the types of design patterns.

Creational Patterns:

1- **Factory Pattern** : In Factory pattern, we create object without exposing the creation logic to the client and refer to newly created object using a common interface.

Usage of Factory Design Pattern **(loose-coupling)**

- When a class doesn't know what sub-classes will be required to create
- When a class wants that its sub-classes specify the objects to be created.
- When the parent classes choose the creation of objects to its sub-classes.

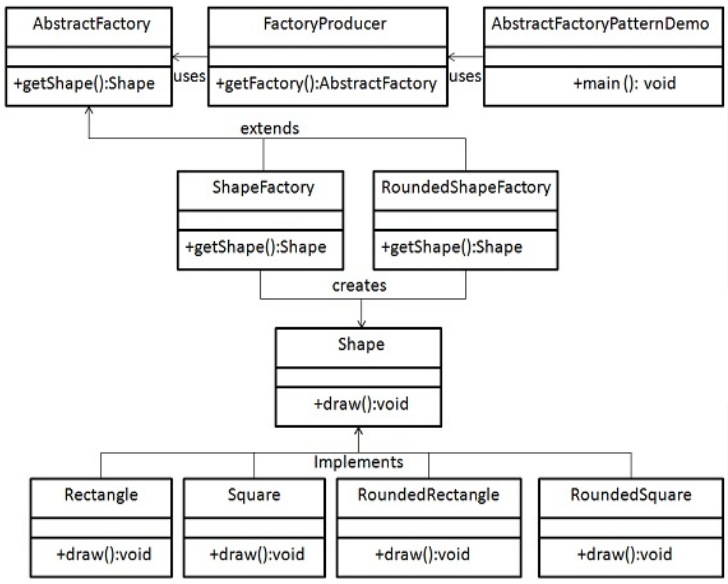


2- **Abstract Factory Pattern :** Abstract Factory patterns work around a super-factory (producer factory) which creates other factories. This factory is also called as factory of factories. That means Abstract Factory lets a class returns a factory of classes. So, this is the reason that Abstract Factory Pattern is one level higher than the Factory Pattern.

In Abstract Factory pattern an interface is responsible for creating a factory of related objects without explicitly specifying their classes. Each generated factory can give the objects as per the Factory pattern.

Usage of Abstract Factory Pattern

- When the system needs to be independent of how its object are created, composed, and represented.
- When the family of related objects has to be used together, then this constraint needs to be enforced.
- When you want to provide a library of objects that does not show implementations and only reveals interfaces.




```
public class ShapeFactory extends AbstractFactory {
   @Override
   public Shape getShape(String shapeType){
      if(shapeType.equalsIgnoreCase("RECTANGLE")){
         return new Rectangle();
      }else if(shapeType.equalsIgnoreCase("SQUARE")){
         return new Square();
      }
      return null;
   }
}
```

RoundedShapeFactory.java

```
public class RoundedShapeFactory extends AbstractFactory {
   @Override
   public Shape getShape(String shapeType){
      if(shapeType.equalsIgnoreCase("RECTANGLE")){
         return new RoundedRectangle();
      }else if(shapeType.equalsIgnoreCase("SQUARE")){
         return new RoundedSquare();
      }
      return null;
   }
}
```

Step 5

Create a Factory generator/producer class to get factories by passing

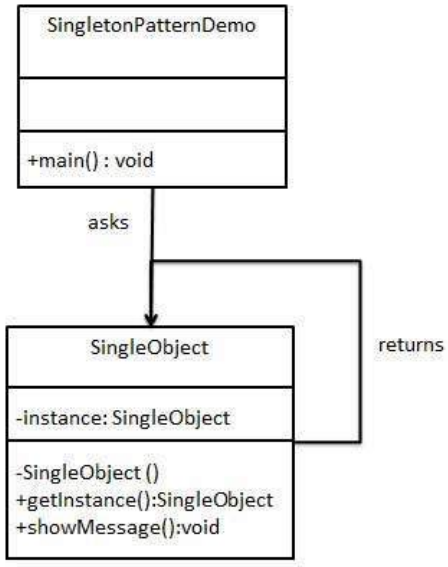
FactoryProducer.java

```
public class FactoryProducer {
   public static AbstractFactory getFactory(boolean rounded
      if(rounded){
         return new RoundedShapeFactory();
      }else{
         return new ShapeFactory();
      }
   }
```

3- **Singleton pattern**: involves a single class which is responsible to create an object while making sure that only single object gets created. Singleton Pattern says that just**"define a class that has only one instance and provides a global point of access to it".** In other words, a class must ensure that only single instance should be created and single object can be used by all other classes. There are two forms of singleton design pattern: **Early Instantiation:** creation of instance at load time. **Lazy Instantiation:** creation of instance when required.

- *Usage of Singleton design pattern (save memory- Only single instance is reused again and again)*
- Singleton pattern is mostly used in multi-threaded and database applications. It is used in logging, caching, thread pools, configuration settings etc

We're going to create a *SingleObject* class. *SingleObject* class have its constructor as private(so NO one can instanciate it except itself) and have a static instance of itself. *SingleObject* class provides a static method to get its static instance to outside world.




```java
public class SingleObject {

   //create an object of SingleObject
   private static SingleObject instance = new SingleObject();

   //make the constructor private so that this class cannot be
   //instantiated
   private SingleObject(){}

   //Get the only object available
   public static SingleObject getInstance(){
      return instance;
   }

   public void showMessage(){
      System.out.println("Hello World!");
   }
}
```

4- **Builder Pattern**: **"construct a complex object from simple objects using step-by-step approach".** It is mostly used when object can't be created in single step.

Advantage of Builder Design Pattern

The main advantages of Builder Pattern are as follows:

- It provides clear separation between the construction and representation of an object.
- It provides better control over construction process.
- It supports to change the internal representation of objects.



5- Prototype Pattern: refers to creating duplicate object while keeping performance in mind. This type of design pattern comes under creational pattern as this pattern provides one of the best ways to create an object.

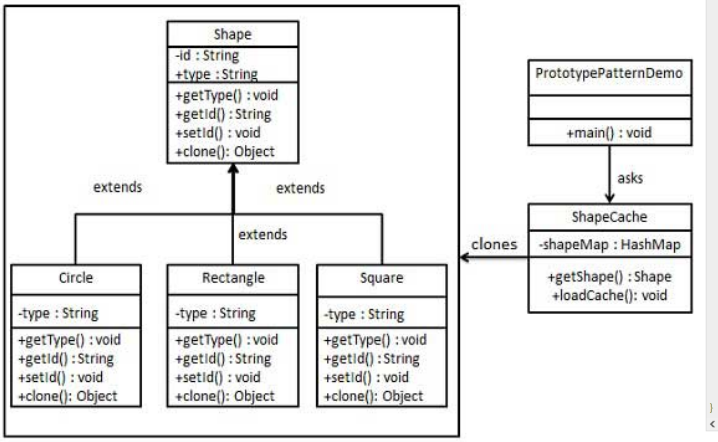
This pattern involves implementing a prototype interface which tells to create a clone of the current object. This pattern is used when creation of object directly is costly. For example, an object is to be created after a costly database operation. We can cache the object, returns its clone on next request and update the database as and when needed thus reducing database calls.

Usage of Prototype Pattern

- When the classes are instantiated at runtime.
- When the cost of creating an object is expensive or complicated.
- When you want to keep the number of classes in an application minimum.
- When the client application needs to be unaware of object creation and representation.

Implementation

We're going to create an abstract class *Shape* and concrete classes extending the *Shape* class. A class *ShapeCache* is defined as a next step which stores shape objects in a *Hashtable* and returns their clone when requested.



ShapeCache.java

```java
import java.util.Hashtable;

public class ShapeCache {

   private static Hashtable<String, Shape> shapeMap  = new Hashtable<String, Sh

   public static Shape getShape(String shapeId) {
      Shape cachedShape = shapeMap.get(shapeId);
      return (Shape) cachedShape.clone();
   }

   // for each shape run database query and create shape
   // shapeMap.put(shapeKey, shape);
   // for example, we are adding three shapes

   public static void loadCache() {
      Circle circle = new Circle();
      circle.setId("1");
      shapeMap.put(circle.getId(),circle);

      Square square = new Square();
      square.setId("2");
      shapeMap.put(square.getId(),square);

      Rectangle rectangle = new Rectangle();
      rectangle.setId("3");
      shapeMap.put(rectangle.getId(), rectangle);
   }
}
```

Structural Patterns:

1- **Adapter Pattern**: Adapter pattern works as a bridge between two incompatible interfaces. This type of design pattern comes under structural pattern as this pattern combines the capability of two independent interfaces. An Adapter Pattern says that just **"converts the interface of a class into another interface that a client wants".**

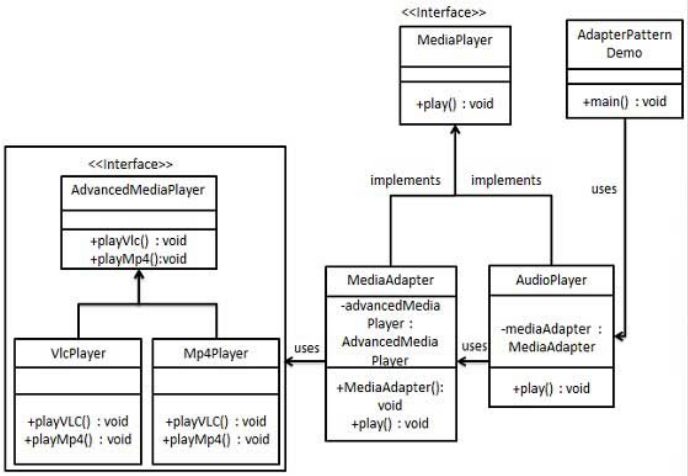
This pattern involves a single class which is responsible to join functionalities of independent or incompatible interfaces. A real life example could be a case of card reader which acts as an adapter between memory card and a laptop. You plugin the memory card into card reader and card reader into the laptop so that memory card can be read via laptop.

Usage of Adapter pattern:

- It allows two or more previously incompatible objects to interact and allows reusability of existing functionality.
- When an object needs to utilize an existing class with an incompatible interface.
- When you want to create a reusable class that cooperates with classes which don't have compatible interfaces.

Implementation

We have a MediaPlayer interface and a concrete class AudioPlayer implementing the MediaPlayer interface. AudioPlayer can play mp3 format audio files by default. We are having another interface AdvancedMediaPlayer and concrete classes implementing the AdvancedMediaPlayer interface. These classes can play vlc and mp4 format files. We want to make AudioPlayer to play other formats as well. To attain this, we have created an adapter class MediaAdapter which implements the MediaPlayer interface and uses AdvancedMediaPlayer objects to play the required format. AudioPlayer uses the adapter class MediaAdapter passing it the desired audio type without knowing the actual class which can play the desired format.



MediaAdapter.java

```java
public class MediaAdapter implements MediaPlayer {

   AdvancedMediaPlayer advancedMusicPlayer;

   public MediaAdapter(String audioType){

      if(audioType.equalsIgnoreCase("vlc") ){
         advancedMusicPlayer = new VlcPlayer();

      }else if (audioType.equalsIgnoreCase("mp4")){
         advancedMusicPlayer = new Mp4Player();
      }
   }

   @Override
   public void play(String audioType, String fileName) {

      if(audioType.equalsIgnoreCase("vlc")){
         advancedMusicPlayer.playVlc(fileName);
      }
      else if(audioType.equalsIgnoreCase("mp4")){
         advancedMusicPlayer.playMp4(fileName);
      }
   }
}
```

2- **Decorator pattern**: allows a user to add new functionality to an existing object without altering its structure. This type of design pattern comes under structural pattern as this pattern acts as a wrapper to existing class. A Decorator Pattern says that just **"attach a flexible additional responsibilities to an object dynamically".** The Decorator Pattern uses composition instead of inheritance to extend the functionality of an object at runtime.

Advantage of Decorator Pattern

- It enhances the extensibility of the object, because changes are made by coding new classes.
- It simplifies the coding by allowing you to develop a series of functionality from targeted classes instead of coding all of the behavior into the object.

Usage of Decorator Pattern

- When you want to transparently and dynamically add responsibilities to objects without affecting other objects.
- When you want to add responsibilities to an object that you may want to change in future.
- Extending functionality by sub-classing is no longer practical.

Implementation

We're going to create a Shape interface and concrete classes implementing the Shape interface. We will then create an abstract decorator class **ShapeDecorator** implementing the Shape interface and having Shape object as its instance variable. RedShapeDecorator is concrete class implementing ShapeDecorator.

```
public abstract class ShapeDecorator implements Shape {
   protected Shape decoratedShape;

   public ShapeDecorator(Shape decoratedShape){
      this.decoratedShape = decoratedShape;
   }

   public void draw(){
      decoratedShape.draw();
   }
}
```

Step 4

Create concrete decorator class extending the *ShapeDecorator* class.

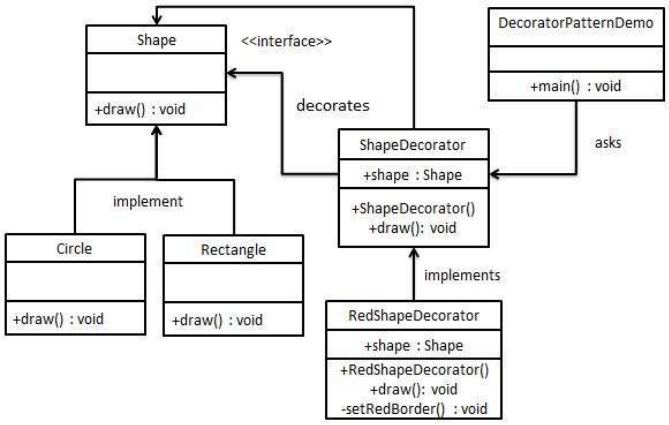
RedShapeDecorator.java

```
public class RedShapeDecorator extends ShapeDecorator {

   public RedShapeDecorator(Shape decoratedShape) {
      super(decoratedShape);
   }

   @Override
   public void draw() {
      decoratedShape.draw();
      setRedBorder(decoratedShape);
   }

   private void setRedBorder(Shape decoratedShape){
      System.out.println("Border Color: Red");
   }
}
```



Step 5

Use the *RedShapeDecorator* to decorate *Shape* objects.

DecoratorPatternDemo.java

```
public class DecoratorPatternDemo {
   public static void main(String[] args) {

      Shape circle = new Circle();

      Shape redCircle = new RedShapeDecorator(new Circle());
```

4- **Composite pattern:** is used where we need to treat a group of objects in similar way as a single object. Composite pattern composes objects in term of a tree structure to represent part as well as whole hierarchy. This type of design pattern comes under structural pattern as this pattern creates a tree structure of group of objects. This pattern creates a class that contains group of its own objects. This class provides ways to modify its group of same objects. We are demonstrating use of composite pattern via following example in which we will show employees hierarchy of an organization.

Usage of Composite Pattern

- When you want to represent a full or partial hierarchy of objects.
- When the responsibilities are needed to be added dynamically to the individual objects without affecting other objects. Where the responsibility of object may vary from time to time.

Implementation

We have a class Employee which acts as composite pattern actor class.

```
public class CompositePatternDemo {
   public static void main(String[] args) {

      Employee CEO = new Employee("John","CEO", 30000);

      Employee headSales = new Employee("Robert","Head Sales", 20000);

      Employee headMarketing = new Employee("Michel","Head Marketing", 20000);

      Employee clerk1 = new Employee("Laura","Marketing", 10000);
      Employee clerk2 = new Employee("Bob","Marketing", 10000);

      Employee salesExecutive1 = new Employee("Richard","Sales", 10000);
      Employee salesExecutive2 = new Employee("Rob","Sales", 10000);

      CEO.add(headSales);
      CEO.add(headMarketing);

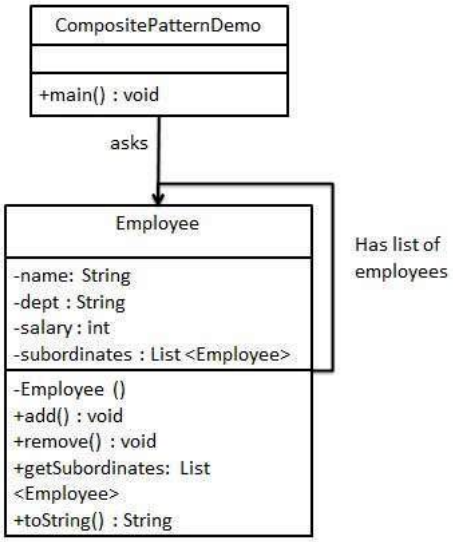
      headSales.add(salesExecutive1);
      headSales.add(salesExecutive2);

      headMarketing.add(clerk1);
      headMarketing.add(clerk2);

      //print all employees of the organization
      System.out.println(CEO);

      for (Employee headEmployee : CEO.getSubordinates()) {
         System.out.println(headEmployee);

         for (Employee employee : headEmployee.getSubordinates()) {
            System.out.println(employee);
         }
      }
   }
}
```



3- Behavioral Design Patterns:

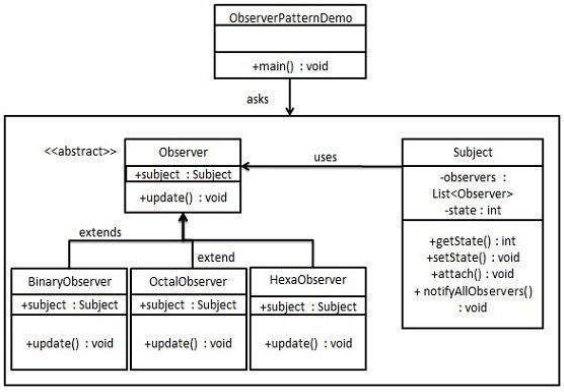
1- Observer pattern: is used when there is one-to-many relationship between objects such as if one object is modified, its depenedent objects are to be notified automatically. Observer pattern falls under behavioral pattern category. An Observer Pattern says that "just define a one-to-one dependency so that when one object changes state, all its dependents are notified and updated automatically". The observer pattern is also known as Dependents or Publish-Subscribe.

Usage:

- When the change of a state in one object must be reflected in another object without keeping the objects tight coupled.
- It provides the support for broadcast-type communication.
- When the framework we writes and needs to be enhanced in future with new observers with minimal chamges.

Implementation

Observer pattern uses three actor classes. Subject, Observer and Client. Subject is an object having methods to attach and detach observers to a client object. We have created an abstract class Observer and a concrete class Subject that is extending class Observer.




```java
import java.util.ArrayList;
import java.util.List;

public class Subject {

   private List<Observer> observers = new ArrayList<Observer>();
   private int state;

   public int getState() {
      return state;
   }

   public void setState(int state) {
      this.state = state;
      notifyAllObservers();
   }

   public void attach(Observer observer){
      observers.add(observer);
   }

   public void notifyAllObservers(){
      for (Observer observer : observers) {
         observer.update();
      }
   }
}
```

```java
public class HexaObserver extends Observer{

   public HexaObserver(Subject subject){
      this.subject = subject;
      this.subject.attach(this);
   }

   @Override
   public void update() {
      System.out.println( "Hex String: " + Integer.toH
   }
}
```

Step 4

Use *Subject* and concrete observer objects.

ObserverPatternDemo.java

```java
public class ObserverPatternDemo {
   public static void main(String[] args) {
      Subject subject = new Subject();

      new HexaObserver(subject);
      new OctalObserver(subject);
      new BinaryObserver(subject);

      System.out.println("First state change: 15");
      subject.setState(15);
      System.out.println("Second state change: 10");
      subject.setState(10);
   }
```

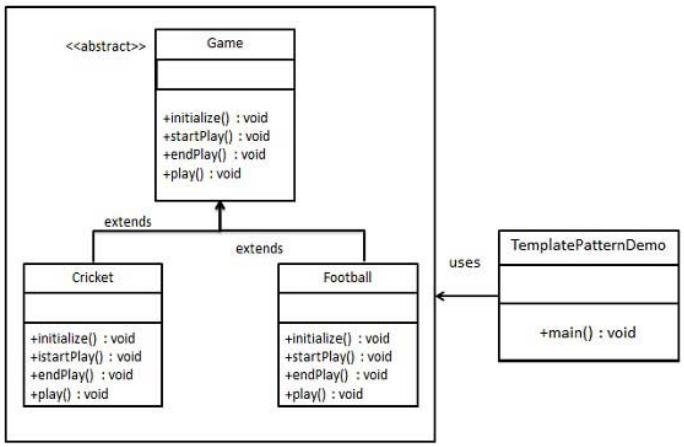
2- Template pattern: an abstract class exposes defined way(s)/template(s) to execute its methods. Its subclasses can override the method implementation as per need but the invocation is to be in the same way as defined by an abstract class. This pattern comes under behavior pattern category. A Template Pattern says that "just define the skeleton of a function in an operation, deferring some steps to its subclasses".

Usage:

- It is used when the common behavior among sub-classes should be moved to a single common class by avoiding the duplication.
- It is very common technique for reusing the code. This is only the main benefit of it.

Implementation

We are going to create a Game abstract class defining operations with a template method set to be final so that it cannot be overridden. Cricket and Football are concrete classes that extend Game and override its methods.




```java
public abstract class Game {
   abstract void initialize();
   abstract void startPlay();
   abstract void endPlay();

   //template method
   public final void play(){

      //initialize the game
      initialize();

      //start game
      startPlay();

      //end game
      endPlay();
   }
}
```

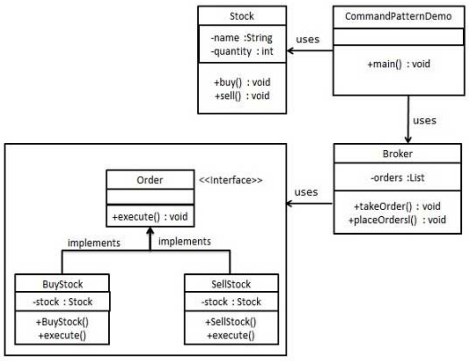
3- Command pattern is a data driven design pattern and falls under behavioral pattern category. A request is wrapped under an object as command and passed to invoker object. Invoker object looks for the appropriate object which can handle this command and passes the command to the corresponding object which executes the command. It is also known as **Action or Transaction.**

Usage of command pattern:

- It is used: It separates the object that invokes the operation from the object that actually performs the operation.
- It makes easy to add new commands, because existing classes remain unchanged.
- When you need parameterize objects according to an action perform.
- When you need to create and execute requests at different times.(event-sourcing –logic)
- When you need to support rollback, logging or transaction functionality.

Implementation

We have created an interface Order which is acting as a command. We have created a Stock class which acts as a request. We have concrete command classes BuyStock and SellStock implementing Order interface which will do actual command processing. A class Broker is created which acts as an invoker object. It can take and place orders.



Broker object uses command pattern to identify which object will execute which command based on the type of command.

```
import java.util.ArrayList;
import java.util.List;

   public class Broker {
   private List<Order> orderList = new ArrayList<Order>();

   public void takeOrder(Order order){
      orderList.add(order);
   }

   public void placeOrders(){

      for (Order order : orderList) {
         order.execute();
      }
      orderList.clear();
   }
}
```

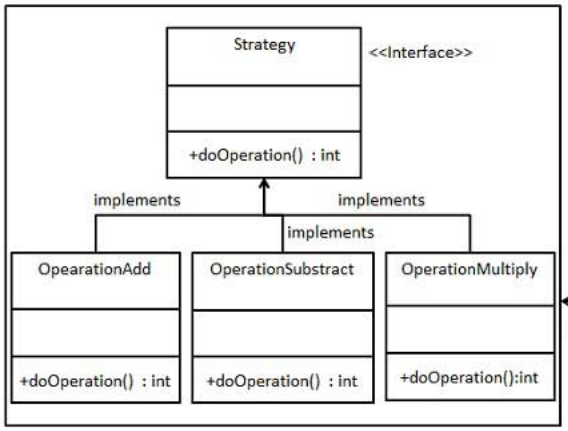
4 Strategy pattern: a class behavior or its algorithm can be changed at run time.(combarable) This type of design pattern comes under behavior pattern. In Strategy pattern, we create objects which represent various strategies and a context object whose behavior varies as per its strategy object. The strategy object changes the executing algorithm of the context object. A Strategy Pattern says that "defines a family of functionality, encapsulate each one, and make them interchangeable".

Usage:

- It is very common technique for reusing the code. This is only the main benefit of it.
- It provides a substitute to subclassing.
- It makes it easier to extend and incorporate new behavior without changing the application.
- When the multiple classes differ only in their behaviors.e.g. Servlet API./ Comparable
- It is used when you need different variations of an algorithm.

Implementation
We are going to create a *Strategy* interface defining an action and concrete strategy classes implementing the *Strategy* interface. *Context* is a class which uses a Strategy.

StrategyPatternDemo, our demo class, will use *Context* and strategy objects to demonstrate change in Context behaviour based on strategy it deploys or uses.




```java
public class OperationMultiply implements Strategy{
   @Override
   public int doOperation(int num1, int num2) {
      return num1 * num2;
   }
}
```

Step 3

Create *Context* Class.

Context.java

```java
public class Context {
   private Strategy strategy;

   public Context(Strategy strategy){
      this.strategy = strategy;
   }

   public int executeStrategy(int num1, int num2){
      return strategy.doOperation(num1, num2);
   }
}
```

Step 4

Use the *Context* to see change in behaviour when it changes its *Strategy*.

StrategyPatternDemo.java

```java
public class StrategyPatternDemo {
   public static void main(String[] args) {
      Context context = new Context(new OperationAdd());
      System.out.println("10 + 5 = " + context.executeStrategy(10, 5));

      context = new Context(new OperationSubstract());
      System.out.println("10 - 5 = " + context.executeStrategy(10, 5));

      context = new Context(new OperationMultiply());
      System.out.println("10 * 5 = " + context.executeStrategy(10, 5));
   }
```

SOLID Princibles:

- The **S**ingle Responsibility Principle
- The **O**pen-Closed Principle
- The **L**iskov Substitution Principle
- The **I**nterface Segregation Principle
- The **D**ependency Inversion Principle

The Single Responsibility Principle

SRP states that **a class should do one thing and therefore it should have only a single reason to change**.

To state this principle more technically: Only one potential change (database logic, logging logic, and so on.) in the software's specification should be able to affect the specification of the class.

This means that if a class is a data container, like a Book class or a Student class, and it has some fields regarding that entity, it should change only when we change the data model.

Following the Single Responsibility Principle is important. First of all, because many different teams can work on the same project and edit the same class for different reasons, this could lead to incompatible modules.

Second, it makes version control easier. For example, say we have a persistence class that handles database operations, and we see a change in that file in the GitHub commits. By following the SRP, we will know that it is related to storage or database-related stuff.

Merge conflicts are another example. They appear when different teams change the same file. But if the SRP is followed, fewer conflicts will appear – files will have a single reason to change, and conflicts that do exist will be easier to resolve.

Common Pitfalls and Anti-patterns

In this section we will look at some common mistakes that violate the Single Responsibility Principle. Then we will talk about some ways to fix them.

We will look at the code for a simple bookstore invoice program as an example. Let's start by defining a book class to use in our invoice.

```
class Book {
    String name;
    String authorName;
    int year;
    int price;
    String isbn;

    public Book(String name, String authorName, int year, int price, String isbn) {
        this.name = name;
        this.authorName = authorName;
        this.year = year;
        this.price = price;
        this.isbn = isbn;
    }
}
```

This is a simple book class with some fields. Nothing fancy. I am not making fields private so that we don't need to deal with getters and setters and can focus on the logic instead.

Now let's create the invoice class which will contain the logic for creating the invoice and calculating the total price. For now, assume that our bookstore only sells books and nothing else.

```
public class Invoice {

    private Book book;
    private int quantity;
    private double discountRate;
    private double taxRate;
    private double total;

    public Invoice(Book book, int quantity, double discountRate, double taxRate) {
        this.book = book;
        this.quantity = quantity;
        this.discountRate = discountRate;
        this.taxRate = taxRate;
        this.total = this.calculateTotal();
    }

    public double calculateTotal() {
            double price = ((book.price - book.price * discountRate) * this.quantity);

        double priceWithTaxes = price * (1 + taxRate);

        return priceWithTaxes;
    }

    public void printInvoice() {
            System.out.println(quantity + "x " + book.name + " " +          book.price + "
            System.out.println("Discount Rate: " + discountRate);
            System.out.println("Tax Rate: " + taxRate);
            System.out.println("Total: " + total);
    }

        public void saveToFile(String filename) {
    // Creates a file with given name and writes the invoice
    }

}
```

Here is our invoice class. It also contains some fields about invoicing and 3 methods:

- **calculateTotal** method, which calculates the total price,
- **printInvoice** method, that should print the invoice to console, and
- **saveToFile** method, responsible for writing the invoice to a file.

You should give yourself a second to think about what is wrong with this class design before reading the next paragraph.

Ok so what's going on here? Our class violates the Single Responsibility Principle in multiple ways.

The first violation is the **printInvoice** method, which contains our printing logic. The SRP states that our class should only have a single reason to change, and that reason should be a change in the invoice calculation for our class.

But in this architecture, if we wanted to change the printing format, we would need to change the class. This is why we should not have printing logic mixed with business logic in the same class.

There is another method that violates the SRP in our class: the **saveToFile** method. It is also an extremely common mistake to mix persistence logic with business logic.

Don't just think in terms of writing to a file – it could be saving to a database, making an API call, or other stuff related to persistence.

So how can we fix this print function, you may ask.

We can create new classes for our printing and persistence logic so we will no longer need to modify the invoice class for those purposes.

We create 2 classes, **InvoicePrinter** and **InvoicePersistence,** and move the methods.

```
public class InvoicePrinter {
    private Invoice invoice;

    public InvoicePrinter(Invoice invoice) {
        this.invoice = invoice;
    }

    public void print() {
        System.out.println(invoice.quantity + "x " + invoice.book.name + " " + invoice.boo
        System.out.println("Discount Rate: " + invoice.discountRate);
        System.out.println("Tax Rate: " + invoice.taxRate);
        System.out.println("Total: " + invoice.total + " $");
    }
}
```

```
public class InvoicePersistence {
    Invoice invoice;

    public InvoicePersistence(Invoice invoice) {
        this.invoice = invoice;
    }

    public void saveToFile(String filename) {
        // Creates a file with given name and writes the invoice
    }
}
```

Now our class structure obeys the Single Responsibility Principle and every class is responsible for one aspect of our application. Great!

Open-Closed Principle

The Open-Closed Principle requires that **classes should be open for extension and closed to modification.**

Modification means changing the code of an existing class, and extension means adding new functionality.

So what this principle wants to say is: We should be able to add new functionality without touching the existing code for the class. This is because whenever we modify the existing code, we are taking the risk of creating potential bugs. So we should avoid touching the tested and reliable (mostly) production code if possible.

But how are we going to add new functionality without touching the class, you may ask. It is usually done with the help of interfaces and abstract classes.

Now that we have covered the basics of the principle, let's apply it to our Invoice application.

Let's say our boss came to us and said that they want invoices to be saved to a database so that we can search them easily. We think okay, this is easy peasy boss, just give me a second!

We create the database, connect to it, and we add a save method to our **InvoicePersistence** class:

```
public class InvoicePersistence {
    Invoice invoice;

    public InvoicePersistence(Invoice invoice) {
        this.invoice = invoice;
    }

    public void saveToFile(String filename) {
        // Creates a file with given name and writes the invoice
    }

    public void saveToDatabase() {
        // Saves the invoice to database
    }
}
```

Unfortunately we, as the lazy developer for the book store, did not design the classes to be easily extendable in the future. So in order to add this feature, we have modified the **InvoicePersistence** class.

If our class design obeyed the Open-Closed principle we would not need to change this class.

So, as the lazy but clever developer for the book store, we see the design problem and decide to refactor the code to obey the principle.

```
interface InvoicePersistence {

    public void save(Invoice invoice);
}
```

We change the type of **InvoicePersistence** to Interface and add a save method. Each persistence class will implement this save method.

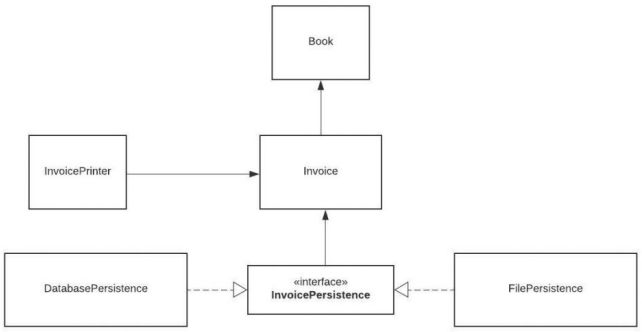
```
public class DatabasePersistence implements InvoicePersistence {

    @Override
    public void save(Invoice invoice) {
        // Save to DB
    }
}
```

```
public class FilePersistence implements InvoicePersistence {

    @Override
    public void save(Invoice invoice) {
        // Save to file
    }
}
```

So our class structure now looks like this:



Now our persistence logic is easily extendable. If our boss asks us to add another database and have 2 different types of databases like MySQL and MongoDB, we can easily do that.

You may think that we could just create multiple classes without an interface and add a save method to all of them.

But let's say that we extend our app and have multiple persistence classes like **InvoicePersistence**, **BookPersistence** and we create a **PersistenceManager** class that manages all persistence classes:

```
public class PersistenceManager {
    InvoicePersistence invoicePersistence;
    BookPersistence bookPersistence;

    public PersistenceManager(InvoicePersistence invoicePersistence,
                              BookPersistence bookPersistence) {
        this.invoicePersistence = invoicePersistence;
        this.bookPersistence = bookPersistence;
    }
}
```

We can now pass any class that implements the **InvoicePersistence** interface to this class with the help of polymorphism. This is the flexibility that interfaces provide.

Liskov Substitution Principle

The Liskov Substitution Principle states that subclasses should be substitutable for their base classes.

This means that, given that class B is a subclass of class A, we should be able to pass an object of class B to any method that expects an object of class A and the method should not give any weird output in that case.

This is the expected behavior, because when we use inheritance we assume that the child class inherits everything that the superclass has. The child class extends the behavior but never narrows it down.

Therefore, when a class does not obey this principle, it leads to some nasty bugs that are hard to detect.

Liskov's principle is easy to understand but hard to detect in code. So let's look at an example.

```
class Rectangle {
    protected int width, height;

    public Rectangle() {
    }

    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }

    public int getWidth() {
        return width;
    }

    public void setWidth(int width) {
        this.width = width;
    }

    public int getHeight() {
        return height;
    }

    public void setHeight(int height) {
        this.height = height;
    }

    public int getArea() {
        return width * height;
    }
}
```

We have a simple Rectangle class, and a **getArea** function which returns the area of the rectangle.

Now we decide to create another class for Squares. As you might know, a square is just a special type of rectangle where the width is equal to the height.

```
class Square extends Rectangle {
    public Square() {}

    public Square(int size) {
        width = height = size;
    }

    @Override
    public void setWidth(int width) {
        super.setWidth(width);
        super.setHeight(width);
    }

    @Override
    public void setHeight(int height) {
        super.setHeight(height);
        super.setWidth(height);
    }
}
```

Our Square class extends the Rectangle class. We set height and width to the same value in the constructor, but we do not want any client (someone who uses our class in their code) to change height or weight in a way that can violate the square property.

Therefore we override the setters to set both properties whenever one of them is changed. But by doing that we have just violated the Liskov substitution principle.

Let's create a main class to perform tests on the **getArea** function.

```
class Test {

   static void getAreaTest(Rectangle r) {
      int width = r.getWidth();
      r.setHeight(10);
      System.out.println("Expected area of " + (width * 10) + ", got " + r.getArea());
   }

   public static void main(String[] args) {
      Rectangle rc = new Rectangle(2, 3);
      getAreaTest(rc);

      Rectangle sq = new Square();
      sq.setWidth(5);
      getAreaTest(sq);
   }
}
```

Your team's tester just came up with the testing function **getAreaTest** and tells you that your **getArea** function fails to pass the test for square objects.

In the first test, we create a rectangle where the width is 2 and the height is 3 and call **getAreaTest.** The output is 20 as expected, but things go wrong when we pass in the square. This is because the call to **setHeight** function in the test is setting the width as well and results in an unexpected output.

Interface Segregation Principle

Segregation means keeping things separated, and the Interface Segregation Principle is about separating the interfaces.

The principle states that many client-specific interfaces are better than one general-purpose interface. Clients should not be forced to implement a function they do no need.

This is a simple principle to understand and apply, so let's see an example.

```
public interface ParkingLot {

    void parkCar(); // Decrease empty spot count by 1
    void unparkCar(); // Increase empty spots by 1
    void getCapacity(); // Returns car capacity
    double calculateFee(Car car); // Returns the price based on number of hours
    void doPayment(Car car);
}

class Car {

}
```

We modeled a very simplified parking lot. It is the type of parking lot where you pay an hourly fee. Now consider that we want to implement a parking lot that is free.

```java
public class FreeParking implements ParkingLot {

    @Override
    public void parkCar() {

    }

    @Override
    public void unparkCar() {

    }

    @Override
    public void getCapacity() {

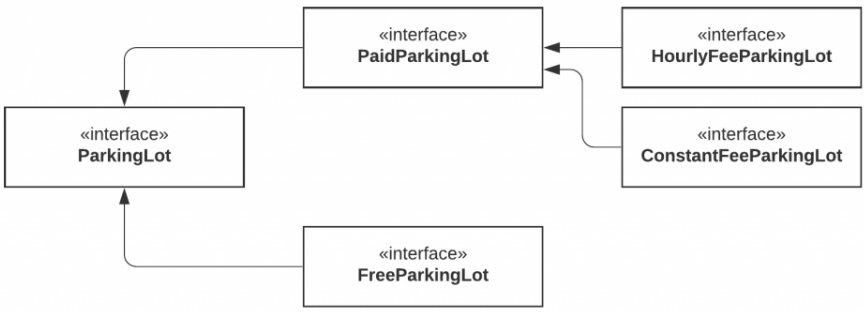
    }

    @Override
    public double calculateFee(Car car) {
        return 0;
    }

    @Override
    public void doPayment(Car car) {
        throw new Exception("Parking lot is free");
    }
}
```

Our parking lot interface was composed of 2 things: Parking related logic (park car, unpark car, get capacity) and payment related logic.

But it is too specific. Because of that, our FreeParking class was forced to implement payment-related methods that are irrelevant. Let's separate or segregate the interfaces.



We've now separated the parking lot. With this new model, we can even go further and split the **PaidParkingLot** to support different types of payment.

Now our model is much more flexible, extendable, and the clients do not need to implement any irrelevant logic because we provide only parking-related functionality in the parking lot interface.

Dependency Inversion Principle

The Dependency Inversion principle states that our classes should depend upon interfaces or abstract classes instead of concrete classes and functions.

In his article (2000), Uncle Bob summarizes this principle as follows:

"If the OCP states the goal of OO architecture, the DIP states the primary mechanism".

These two principles are indeed related and we have applied this pattern before while we were discussing the Open-Closed Principle.

We want our classes to be open to extension, so we have reorganized our dependencies to depend on interfaces instead of concrete classes. Our PersistenceManager class depends on InvoicePersistence instead of the classes that implement that interface.

Architectural patterns:

Event Sourcing vs Event Driven Design

Event driven architectures

The term Event driven architecture is used for any kind of software system which is based on components communicating mainly or exclusively through events. This software architecture paradigm promotes the production, detection, consumption of, and reaction to events. An event can be defined as “a significant change in state”. In the context of EDAs, the term “event” usually means “notification”. [2]

Event sourcing

Event sourcing is a much more special term, referring to systems that store all changes to the application state as a sequence of events. A well-known popular class of examples are transactional database systems, which store any state changes in a transaction log. Here, the term “event” refers more to “state change”, not only to “notification”.

The state is persisted as a series of events in a place called the *event store*. New events get appended, but they don’t overwrite old events. As a result, all history is maintained.

Comparison

Let’s contrast them in several areas:

Purpose

Event sourcing is an alternative to traditional persistence strategies, with the purpose of keeping history. Event driven architecture is a distributed asynchronous architecture pattern used to produce highly scalable applications. In other words, event sourcing is about having historical records in an organisation, whereas EDAs are highly adaptable, scalable and can be used for small applications and as well as large, complex ones.

Scope

Event sourcing is usually applied in a single application/system, maybe implemented as services; however usually sharing the same storage. Meanwhile, EDAs are used across several apps/systems. Being a reliable integration pattern to achieve high levels of agility to respond quickly to a constantly changing environment.

Data storage

Event sourcing has a central event store, which usually has replicas, sharding, etc. It relies on a central database. By contrast, EDAs are distributed so each component (or event processors) is decoupled and most likely to have their own storage.

Testability

Event sourcing is easier to test in comparison, since it can replay the whole sequence of events from scratch to arrive to a certain state/situation. On the other hand, EDAs are easy to test for each component in isolation, however testing the whole is complicated by the asynchronous nature of this pattern.

Event driven architectures vs event sourcing

Aspects	Event sourcing pattern	Event Driven Architecture pattern
Purpose	Keeping history	Highly adaptable and scalable
Scope	Single application/system	Whole organisation/several apps
Storage	Central event store	Decentralised
Testing	Easier	Harder

Monolithic Architecture

The term monolith described objects made from a single large piece of material. In software development, monolithic architecture is when the whole application has a single code base and builds to one executable program. To make everything more manageable, engineers divide the application by a business or technical features into smaller separate modules. Unlike in service-oriented architecture, the modules communicate with each other directly by calling functions.

Pros of a Monolith

Simplicity

While a monolithic application can quickly turn into a big bowl of spaghetti code, the architecture itself aims for simplicity. Programmers import functions from different modules, and that's it. Everything runs in the same app process — no timeouts, crashes, latency, different module versions, and the majority of the code is synchronous. The deployment also boils downs to updating a program and restarting it.

Performance and Latency
Calling a function in memory is much faster than making a network call and waiting for the results. The data in RAM is accessible by the whole application, and no network hops are needed. While an app with microservice architecture might have to make 20 different API calls to various services that, in turn, do more API calls and database requests, a monolith only has to make a couple of database requests to accomplish the same task.

Codesharing and Refactoring
Everything is in the same codebase and get's built as a whole. Thus, developers can find and understand all places where a particular piece of code is used. When refactoring or removing unused code, they can be sure they won't break something that possibly depends on it. They also don't have to deal with multiple versions of the same shared code deployed simultaneously.

Debugging
Debugging a monolith application is pretty straightforward. Just run the application with the debugger enabled. You don't have to prepare a dedicated debugging environment, attach to multiple services, and track what's happening where. Everything is in a single program that runs on the developer's computer.

Cons of a Monolith

Scalability
Though a monolith can be scaled horizontally, it's usually not that straightforward. It might have some state in RAM or rely on exclusive access to the database. Even when appropriately designed, all modules will be running on every server. You can't just scale a part of the application that needs more resources.

Reliability
When an application crashes, everything goes down. A bug in a single module can bring the whole system down. When one part of the application eats all the resources, every other aspect of it suffers too. It's pretty hard to create a highly available, scalable, fault-tolerant monolith.

Tight Coupling
Maintaining a clean codebase when a project grows becomes difficult. Developers always tend to make a global variable, a singleton, or make a piece of internals public. It's too easy when you have all the code in front of you.

Single Technology Stack
With a monolith, you're tied to a single platform and possibly programming language. Some platforms, like .Net and Java, do allow almost complete codesharing between modules in different languages. It isn't easy to adopt new technologies that might be better suited for future challenges.

Limited Agility
When several teams are working together on a project, codesharing becomes an issue. Engineers often have to resolve merge conflicts and check if they haven't broken anything with their code in a code that other people wrote. Deployments have to be thoughtfully planned, and the whole team should wait for them to see their code running in production.

Microservice Architecture

The applications built with a service-oriented architecture (SOA) consist of many loosely coupled services that communicate with each other over a network. Microservice architecture is a variant of SOA with

tiny services using lightweight protocols. Each microservice is independent, with ownership of its dedicated databases and other resources.

Pros of Microservices

Scalability

You can scale each service separately according to the required resource usage and the current load. While one service can run on hundreds of instances, the other can be idling running on just a single machine. Best engineers will even design the services according to their planned load. A reporting service used only by the product team can use simple relational databases and queries on them. In contrast, the user-facing part of the application would use fast No-SQL databases and some caching.

Agility

With good design, there's no need to deploy the whole application all at once. Each service can be updated separately without affecting the other services, making continuous delivery possible and reducing the entire application's development-deployment cycle duration.

Reliability

If one of the services goes down, it doesn't affect the whole application. Yes, the functionality under its responsibility won't be available and may affect the connected services, but most of an extensive application will usually remain working.

Access to New Technologies

Unlike the monolith, separate services can leverage different technologies or even programming languages. They can even run on different operating systems. The possibilities are endless as long as every service is still able to understand each other.

Simplicity of Services

Every separate microservice usually has only a few thousand lines of code. Thus it's easy to read and understand all the nuts and bolts. Debugging the service is a breeze. Unit tests are also more straightforward as code is loosely coupled. The onboarding process for new developers is concise and takes hours instead of days or weeks.

Parallel Development

Due to loose coupling and independence of the services, it's easy to work on them in parallel. Just think through and design the API interaction points so that different developers or even teams can build them simultaneously. Engineers can temporarily create mocks for the necessary APIs and switch back to the real services once those ready.

Reusability

The services are pretty self-contained to re-use them even in a different application. For example, multiple applications might share the shopping cart, and authentication services are often pretty similar.

Cons of Microservices

Transactional Complexity

Adding an item to inventory in monolith was as simple as updating a few database records in a single transaction. It's no longer that simple in the microservice world. The inventory service should check the inventory and probably reserve some. The shopping card service should add an item to the card. The

related product service should check for relevant suggestions, and multiple analytics services should react to the user action. And what if some of the operations fail? Something goes down? Does the timeout mean it failed, or it's gone be done? When should we restore the inventory? There are approaches to handle this, but they're still much more complicated than in a simple monolith.

Debugging Complexity
When debugging a single interaction in a microservice application, developers must run multiple services locally or attach to them remotely. Tracing back the data and matching different services logs to understand why something happened becomes quite a tricky task. There aren't many tools to help with that either.

Deployment Complexity
Everyone can run the monolithic application by merely executing the entry point file. In contrast, microservices always require some configuration to know about each other and find the correct dependency services. There's scalability, fault tolerance, load balancing, etc. You'll need a DevOps for even moderately complex applications to get all the benefits of this architecture.

Increased Latency
In a monolith, most of the code runs in a single process. In microservice applications, everything is done asynchronously on different machines. These distributed systems have higher latency because they need many more network hops to finish the same task. Moreover, when using an event queue to simplify things, the events may build up under heavy load, drastically decreasing the system's performance.

Increased Attack Surface
With a monolith, only the public API is accessible on the network. With microservices, many services also expose private endpoints for inter-service communication. Those should be carefully guarded against the outside world.

Different Technologies
Using better-suited technologies for specific tasks is a good thing, but it drastically increases the new developers' entry barrier. With different libraries, frameworks, and even programming languages, you'll have hard times onboarding new developers or even need separate non-interchangeable teams for multiple parts of the application. Not all javascript developers also know Python or C++ and vice versa.

Serverless architecture

Serverless is another variant of service-oriented architecture designed specifically for cloud computing. The cloud provider handles many aspects of infrastructure management and scaling automatically, relieving the technical team from provisioning and server maintenance. Of course, the application still runs on servers, but everything is abstracted away so that developers can focus more on business logic. Instead of services, you have to deploy separate functions executed on triggers like HTTP requests, time, or various events.

Serverless shares many pros and cons with the microservice architecture, so we'll focus only on the differences.

Pros of Serverless

Improved Scalability

With serverless, you get entirely automatic scaling. The app will automatically upscale and downscale, depending on the current load, maintaining almost constant performance. These applications can handle huge loads, instantly scaling to thousands of parallel executions. Of course, with most cloud providers, you can also set the limits to stay within the budget.

Easy Deployment

You don't have to worry about the infrastructure. Just push new code to the cloud, and it will be provisioned automatically. The project would need a much smaller DevOps team than microservices, where DevOps are also responsible for a ton of configuration.

Lower Costs

You're developers and DevOps are working on more critical business-oriented tasks. The code is somewhat more straightforward and much more loosely coupled. You're charged for the CPU cycles and RAM used by your application. That reduces both the development and running cost of the project.

Cons of Serverless

Slow Cold Starts

With monolithic architecture or microservices, your application is always ready to handle requests. That's not the case with serverless. Functions that were recently executed stay prepared to handle new requests for quite some time instantly and are called warm. When the operation hasn't been invoked for a while, it gets unloaded by the cloud provider, and spinning it up again induces additional latency. This cold start latency also occurs when scaling. For example, ten instances of the same function might be warm, but the cloud provider would have to spin up to five more cold instances for an increased momentary load. There are techniques to keep the functions warm, but they induce additional cost by calling the API on timers.

Vendor Lock-In

There's no standard for writing serverless functions. Every function should have a cloud provider-specific code to run serverless. The build process, local start scripts, and debugging are also different. There's a Serverless framework that abstracts all of that for the major vendors. However, sometimes you'll still need more fine-grained control over the configuration that's only possible with vendor-specific code.

Platform Restrictions

Since serverless functions have specific build targets and frameworks to run in the cloud, cloud providers have restrictions on the programming languages or platforms they support. The popular ecosystems are usually covered, but you'll have a hard time trying something less mainstream.

Limited Code Reuse

You have to care more about your function's memory usage and startup times. That means carefully choosing what you should include in their code. Writing an extensive shared library with lots of global variables and their initialization code is not a good idea with serverless.

Not for Long-Running Tasks

While you can run serverless functions at regular time intervals, running it for longer than a few seconds is usually not supported. It will be terminated on timeout by the cloud provider. It's also not a good idea

in most cases since you'll be charged for CPU cycles more than for equivalent server instances running full-time.

Many Standalone Functions
Mature serverless projects typically have thousands of separately deployed functions. As the project grows, it gets harder to keep track of what's still used and what's deprecated. Security is also a significant concern since, ideally, you'll need a separate set of permissions for every function. And all of them have their configuration too.

Summary

	Monolith	Microservices	Serverless
of Development			ate
omplexity	epends	ate/Low	epends
ctional Tasks		ate	
sts	epends		
tion Tests		ate	ate
haring		ate	
m Restrictions	uild		ud Provider
ging		ate	
lity			asy
lity			asy
y		ate	
ment Complexity			
nance	ate		
pment Costs	crease in time	onstant in time	crease in time
g Costs		ate	ds

Conclusion
There's rarely one single best architecture for the specific project. There's always some tradeoffs you have to consider. From our experience

- Monolithic architecture is excellent for prototypes, MVPs, and startups in the early stages. The speed of development is essential here, and monolithic architecture, with its simplicity, shines here, making fast changes possible. The team size is small, and there are not that many users either, so those drawbacks aren't that important. Unless the application will realistically face some crazy loads from day one, we would suggest going with monolith for an MVP.
- Microservices are best for applications with a constant load that have to scale horizontally. They're also great for bigger teams thanks to parallel development. If done correctly, a monolithic MVP can be split to microservice quite easily when needed.
- Serverless is helpful for rarely used parts of the application, projects with very inconsistent loads, or adding some back-end functionality to static sites. On paper, it has many benefits to microservices, but in real-world scenarios, you'll pay more for CPU and RAM under constant load than for an equivalent server running full-time. The cold start latency is also something to consider.

Single-tenant vs. multi-tenant design

The *tenancy* is an analogy for exchanging computing resources in a software environment, where tenants represent neighbors living either in the apartment building (or a private household). Now, what are the differences between single-tenant and multi-tenant databases for a SaaS?

In a **single-tenant architecture**, the tenant *owns* an application. Each client uses an isolated database, storage, and web server. Single-tenant design is popular among larger enterprises that want maximal customization and secure solutions. The customer often hosts the app. Consequently, it's up to them to develop, host, and maintain the cloud infrastructure.

The examples of a multi-tenant architecture include such services as:

- **Gmail:** email clients are the prime examples of SaaS that doesn't require setting up and maintenance.
- **Google Drive** and **Dropbox:** cloud file hosting services offer secure and easy-to-access storage over the Internet.

Multi-tenant SaaS providers allow clients to run sophisticated apps and large databases immediately while spending less money. Moreover, tenants don't have to maintain the infrastructure, allowing them to save storage, hardware, and CPU costs. Therefore, it's a perfect option for startups, medium-sized companies, and organizations with smaller IT teams.

Differences between Single-Tier and Multi-Tier solutions

Single-tier architecture implies putting all of the required components for a software application (both the backend and the frontend) on just one server. This is a good way to test your application in development environments and it is an ideal solution for small sites with low traffic demand which require effective resource utilization. It is handy to manage and maintain and, of course, a Single-Tier deployment is cost-effective.

But having all the resources on the same machine can create an availability and security risk. If the server is down, the application will be down, and it will not communicate with the database. If the server is externally attacked, you are at greater risk of data loss if you do not have a replica of your database.

The typical architecture of a Bitnami Single-Tier Solution looks like this:



Multi-tier architecture solves these problems by splitting data access across more than one server. Having all the resources spread into different servers boosts your deployment performance. In addition to this, having different layers for different resources implies adding an extra security layer by separating data from code. In those applications that include replication, the database can be replicated across more than one server which prevents the loss of data in case of cluster failure.

This architecture also provides high scalability and failover: you can add as many nodes as you need to increase the capacity of your cluster. This way, the workload is also decentralized ensuring that when a node is down, the rest of the deployment is working.

All Bitnami Multi-Tier stacks are production ready following industry conventions: you can move your deployments from development to production in an easy and a reliable way. Multi-tier solutions are a perfect choice to ensure high availability and performance in medium/large size production environments.

Software Architecture

Software Architecture consists of One Tier, Two Tier, Three Tier, and N-Tier architectures. A “tier” can also be referred to as a “layer”. Three layers are involved in the application namely Presentation Layer, Business Layer, and Data Layer. Let’s see each layer in detail:

#1. Presentation Layer

It is also known as the Client layer. The top most layer of an application. This is the layer we see when we use the software. The main function of this layer is to communicate with the Application layer. This layer passes the information which is given by the user in terms of keyboard actions, mouse clicks to the Application Layer.
For example, the login page of Gmail where an end-user could see text boxes and buttons to enter user id, password, and to click on sign-in. In simple words, it is to view the application.

#2. Application Layer

It is also known as Business Logic Layer which is also known as the logical layer. As per the Gmail login page example, once the user clicks on the login button, the Application layer interacts with the Database layer and sends required information to the Presentation layer. It controls an application's functionality by performing detailed processing. This layer acts as a mediator between the Presentation and the Database layer. Complete business logic will be written in this layer. In simple words, it is to perform operations on the application.

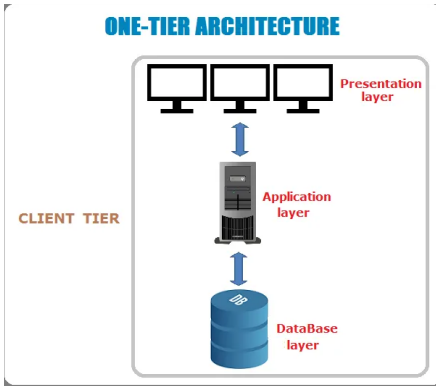
#3. Data Layer

The data is stored in this layer. The application layer communicates with the Database layer to retrieve the data. It contains methods that connect the database and performs required action e.g.: insert, update, delete, etc. In simple words, it is to share and retrieve the data.

Types of Software Architecture:

#1. One Tier Architecture:

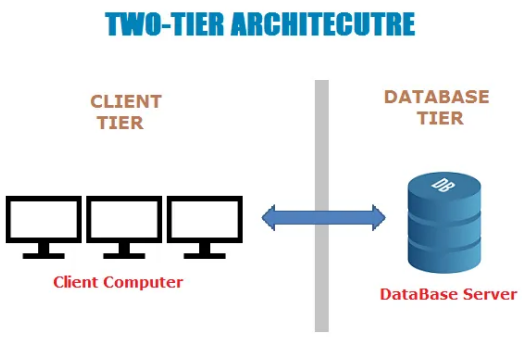
One Tier application AKA Standalone application



One-tier architecture has all the layers such as Presentation, Business, Data Access layers in a single software package. Applications that handle all the three tiers such as MP3 player, MS Office come under the one-tier application. The data is stored in the local system or a shared drive.

#2. Two-Tier Architecture:

Two Tier application AKA Client-Server application



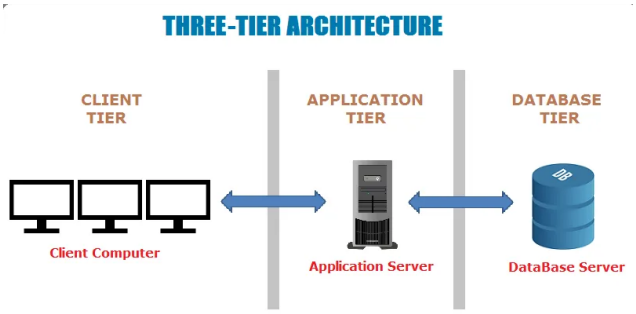
The Two-tier architecture is divided into two parts:

1. Client Application (Client Tier)
2. Database (Data Tier)

The client system handles both Presentation and Application layers and the Server system handles the Database layer. It is also known as a client-server application. The communication takes place between the Client and the Server. The client system sends the request to the server system and the Server system processes the request and sends back the data to the Client System

#3. Three-Tier Architecture:

Three Tier application AKA Web Based application

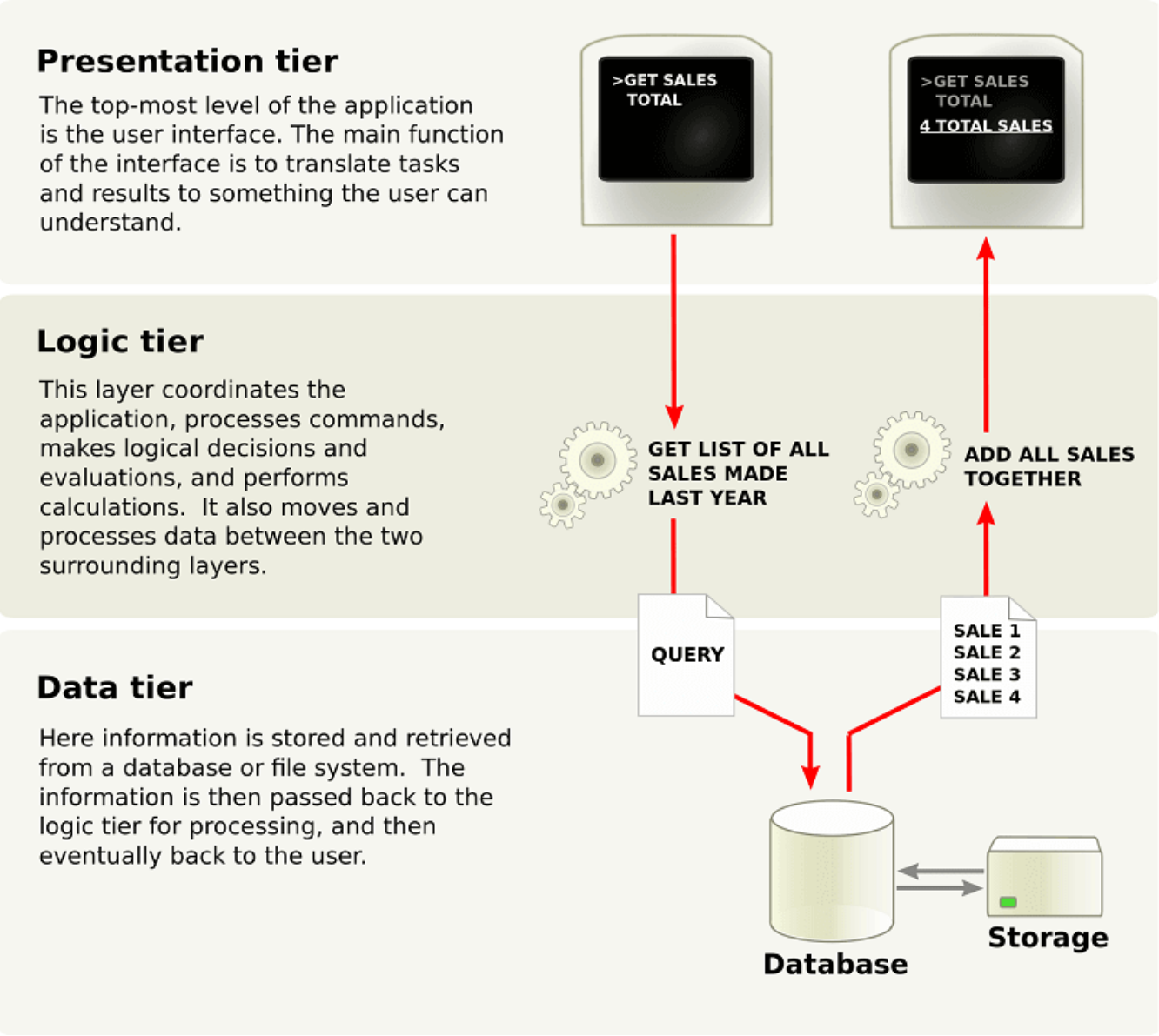


The Three-tier architecture is divided into three parts:

1. Presentation layer (Client Tier)
2. Application layer (Business Tier)
2. Database layer (Data Tier)

The client system handles the Presentation layer, the Application server handles the Application layer, and the Server system handles the Database layer.

Note: Another layer is the N-Tier application. N-Tier application AKA Distributed application. It is similar to the three-tier architecture but the number of application servers is increased and represented in individual tiers in order to distribute the business logic so that the logic will be distributed.



Difference Between Architectural Style, Architectural Patterns and Design Patterns

Many software professionals think that architectural styles and patterns are the same. Sadly, some of the software developers don't understand the difference between architectural patterns and design patterns. In this article, we're going to summarize the differences between them.

Architectural Style

The architectural style shows how do we organize our code, or how the system will look like from 10000 feet helicopter view to show the highest level of abstraction of our system design. Furthermore, when building the architectural style of our system we focus on layers and modules and how they are communicating with each other.

There are different types of architectural styles, and moreover, we can mix them and produce a hybrid style that consists of a mix between two and even more architectural styles. Below is a list of architectural styles and examples for each category:

- **Structure architectural styles:** such as layered, pipes and filters and component-based styles.
- **Messaging styles:** such as Implicit invocation, asynchronous messaging and publish-subscribe style.
- **Distributed systems:** such as service-oriented, peer to peer style, object request broker, and cloud computing styles.
- **Shared memory styles:** such as role-based, blackboard, database-centric styles.
- **Adaptive system styles:** such as microkernel style, reflection, domain-specific language styles.

Architectural Patterns

The architectural pattern shows how a solution can be used to solve a reoccurring problem. In another word, it reflects how a code or components interact with each other. Moreover, the architectural pattern is describing the architectural style of our system and provides solutions for the issues in our architectural style. Personally, I prefer to define architectural patterns as a way to implement our architectural style. For example: how to separate the UI of the data module in our architectural style? How to integrate a third-party component with our system? how many tires will we have in our client-server architecture? Examples of architectural patterns are microservices, message bus, service requester/ consumer, MVC, MVVM, microkernel, n-tier, domain-driven design, and presentation-abstraction-control.

Design Patterns

Design patterns are accumulative best practices and experiences that software professionals used over the years to solve the general problem by – trial and error – they faced during software development.

1. Creational design patterns:
Provide a way to create objects while hiding the creation logic. Thus, the object creation is to be done without instantiating objects directly with the "New" keyword to gives the flexibility to decide which objects need to be created for a given use case. The creational design patterns are:

- **Abstract factory pattern:** provide an interface to create objects without specifying the classes.
- **Singleton pattern:** provide only a single instance of the calls and global access to this instance.
- **Builder Pattern:** Separate the construction from representation and allows the same construction to create multiple representations.
- **Prototype pattern:** creating duplicate without affecting the performance and memory. So the duplicate object is built from the skeleton of an existing object.

2. Structural patterns:

Concerned with class and object composition. The Structural design patterns are:

- **Adapter pattern:** it works as a bridge between two incompatible interfaces and compines their capabilities.
- **Composite pattern:** provide a way to treat a group of objects in a similar way as a single object. It composes objects in term of a tree structure to represent part as well as a whole hierarchy
- **Decorator pattern:** allows adding new functionality to an existing object without altering its structure.

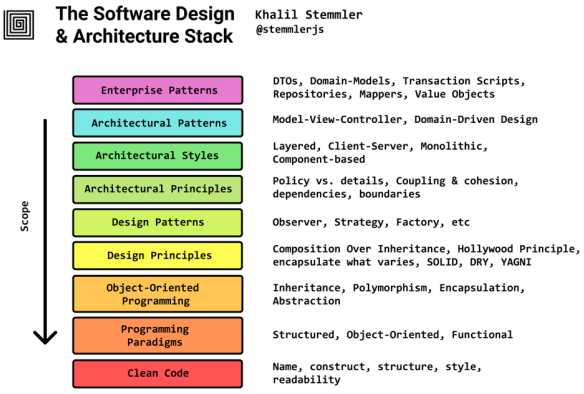
3. Behavioral patterns:

Behavioral patterns are concerned with communications between objects. The following is the list of behavioral patterns:

- **Command pattern:** it's a data-driven pattern in which A request is wrapped under an object as command and passed to an invoker object.
- **Observer pattern:** used when there is a one-to-many relationship between objects such as if one object is modified, its dependent objects are to be notified automatically.
- **Strategy pattern:** provides a way to change class behavior or its algorithm at run time.
- **Template pattern:** an abstract class exposes defined way(s)/template(s) to execute its methods. Its subclasses can override the method implementation as per need but the invocation is to be in the same way as defined by an abstract class.

The bottom line:

The architectural style is a 10000-helicopter view of the system. It shows the system design at the highest level of abstraction. It also shows the high-level module of the application and how these modules are interacting. On the other hand, architectural patterns have a huge impact on system implementation horizontally and vertically. Finally, the design patterns are used to solve localized issues during the implementation of the software. Also, it has a lower impact on the code than the architectural patterns since the design pattern is more concerned with a specific portion of code implementation such as initializing objects and communication between objects.



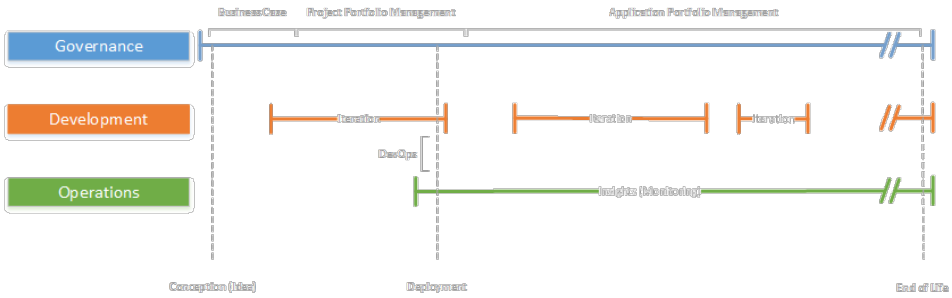
AOP:

AOP is a programming paradigm that aims to increase modularity by allowing the separation of cross-cutting concerns.(Logical or functionality : basic infrastructure code that all application like loggining o security) It does this by adding additional behavior to existing code without modifying the code itself.

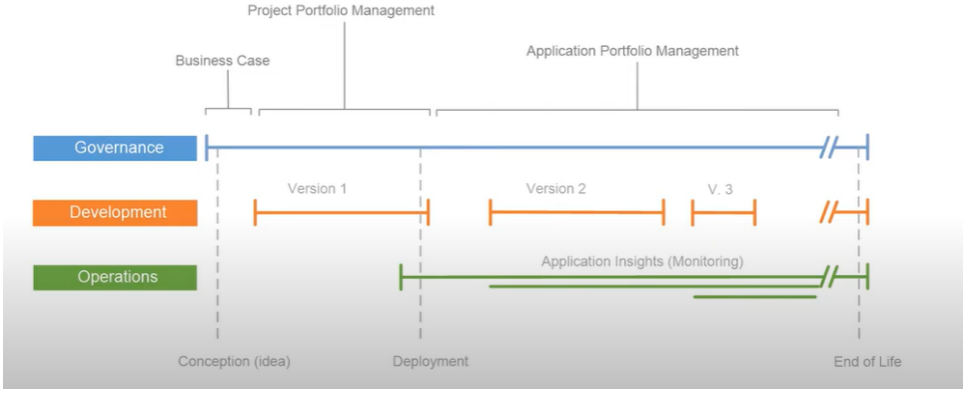
Instead, we can declare the new code and the new behaviors separately.

Alm and Agile

Before Agile:



After:



Agile ALM is the practice of using Agile processes to manage your requirements, issues, and tests.

There are plenty of reasons why development teams implement Agile to manage their project lifecycles.

Customers expect rapid releases of quality software. You can't spend months or years perfecting a release. (But you can't send it out into the market with too many bugs, either.)

Applying Agile to ALM helps you:

- Deliver quality releases quickly.
- Improve collaboration across teams.
- Prioritize customer needs.

Agile is all about value and people. And ALM is all about tools.

Applying Agile Theories…
Understanding Agile ALM begins by looking first at each of the four main principles laid out in the Agile Manifesto.

Agile Software Development (manifesto)

Individuals and interactions over *processes and tools*
Working software over *comprehensive documentation*
Customer collaboration over *contract negotiation*
Responding to change over *following a plan*

Individuals and Interactions Over Processes and Tools.
Traditional ALM is often governed by steps for completing the product. First you gather requirements. Then you build the product. And then you run tests. This can create silos by process. Agile ALM, instead, **favors cross-functional team interaction**. Gathering requirements is a collaborative process — and these requirements are reviewed and updated constantly. The product is built and tested in sprints, facilitating greater collaboration throughout the process.

Working Software Over Comprehensive Documentation.
ALM is traditionally characterized by extensive documentation. Agile suggests prioritizing fast (quality) releases over documentation. Hybrid Agile typically bridges the gap for organizations who need the documentation (e.g., for compliance) but want to be Agile.

Customer Collaboration Over Contract Negotiation.
In traditional ALM, customer feedback comes in the form of requests for bug fixes or features. It usually doesn't inform requirements. In Agile ALM, customer feedback is critical. It's often included in requirements gathering throughout the development lifecycle.

Responding to Change Over Following a Plan.
Traditional ALM is all about the plan. The plan is set firmly at the beginning of development and followed through to deployment.

Agile ALM responds to change. The plan — if you call it that — is in flux(flowing or flowing out). This helps team members be more productive and deliver the right functionality at the right time. And that typically helps you reduce your costs.

Going From Theory to Practice
Agile is straightforward in theory. But it takes work to implement Agile processes. And transitioning from traditional development (e.g., Waterfall) to Agile doesn't happen overnight.

Some teams make the mistaking of trying to become Agile too quickly. This can be disorienting for some team members. And this makes organizational Agile adoption difficult.

Collaborate on Agile Requirements
Agile requirements are different from traditional requirements.

Traditional requirements are set early on in development. Some of these may come from business requirements. While traditional requirements may change during development, they do not change very

often. Traditional requirements clearly outline what the requirements are. But they don't outline why those requirements need to be met.

Agile requirements are set in user stories when they are needed. Stakeholders should actively participate in the requirements gathering process. And user stories should be constantly refined and prioritized.Agile requirements outline what the requirement is and why it matters to the customer, stakeholder, or company.

Do Agile Issue Tracking
Agile issue tracking differs from traditional issue tracking methods.

Traditionally, issues are found by testers and resolved prior to a release. If a customer finds an issue, it's added to a list of pending bug fixes. The issue then gets assigned to someone. At some point, the issue is fixed and tested.

Agile issue tracking invites collaboration to resolve issues. When an issue comes in, it is prioritized based on impact. A team works through this backlog of issues, resolving issues as quickly as they can.

Run Agile Tests
Agile testing is managed differently than traditional testing.

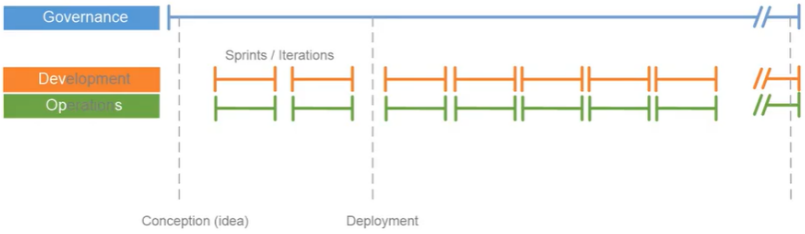
Traditional testing happens after a build is completed. Test cases are defined early on. Most test runs are completed manually. If there are automated tests involved, those testing results are tracked manually too. Testers handle their work individually. Most importantly, traditional test plans are rarely reviewed or changed.

Agile testing happens continuously. Test cases are set based on user stories. As new user stories are added, new test cases should be added too. Testing is done in sprints, often utilizing test automation to increase test coverage. Most importantly, test plans are constantly reviewed and changed to meet changing needs.

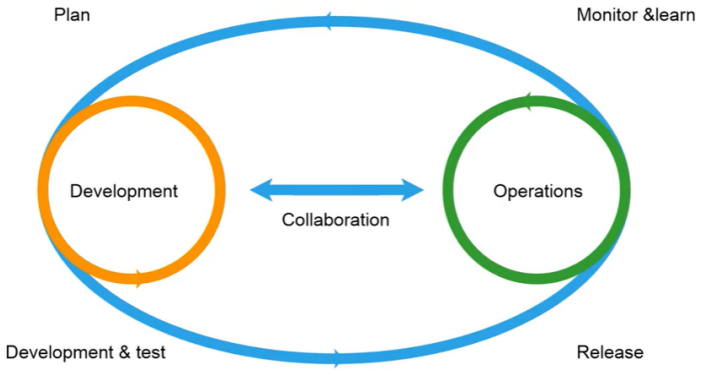
Managing Requirements in Agile Using an ALM Tool
What makes an Agile lifecycle management tool different from a traditional ALM tool?

An Agile lifecycle management tool should be tightly integrated into your development environment. It should be easy for your development teams to collaborate on requirements, issue resolution, and testing. An Agile tool also needs to be easy-to-use. There shouldn't be a big learning curve.

Most of all, an Agile lifecycle management tool should follow Agile ALM best practices. It should support **rapid releases, continuous testing, and Agile planning.**



DevOps Lifecycle



Continuous ...

